

高级语言程序设计

实验报告

南开大学 计算机大类

姓名 傅思敬

学号 2512869

班级 1010

2026 年 5 月 13 日

目录

高级语言程序设计大作业实验报告	2
一. 作业题目	2
二. 开发软件	2
三. 课题要求	2
四. 主要流程	3
五. 单元测试	5
六. 收获	7

高级语言程序设计大作业实验报告

一. 作业题目

galgame-creator ——基于 qt6 开发的视觉小说开发工具

二. 开发软件

编程语言：C++

IDE：trae, qt creator

框架：qt6, cmake

版本控制：git

三. 课题要求

- 1) 面向对象。
- 2) 单元测试。
- 3) 模型部分
- 4) 验证

四. 主要流程

1. 开发主要流程 (Development Process)

项目遵循了软件工程中“需求分析 → 架构设计 → 模块开发 → 集成测试”的标准流程：

- 1. 需求分析与建模：
 - 定义 Galgame 的核心要素：场景（背景图）、角色（立绘、坐标）、对话（文本、语音、背景音乐）。
 - 确立“编辑器”与“播放器”分离的思路：编辑器负责生产 JSON 数据包，播放器负责解析并渲染。
- 2. 核心架构设计 (MVC 模式思路)：
 - Model (数据层)：设计 `Character`、`Scene`、`Project` 等 C++ 类，利用 `QJsonObject` 实现数据的序列化与反序列化。
 - View (界面层)：使用 `QGraphicsView` 搭建可视化画布，实现立绘的自由拖拽预览。
 - Controller (逻辑层)：编写信号槽逻辑，确保属性面板（颜色选择器、坐标微调）的改动能实时同步到模型。
- 3. 播放引擎状态机开发：
 - 开发 `PlayerWindow`，其核心是一个状态机。它根据当前对话索引，依次加载资源、触发打字机效果、播放音频，并监听用户点击或自动播放信号。
- 4. 导出与部署工程：
 - 实现资源自动收集算法，将项目引用的所有外部素材统一打包，并补齐 Qt 运行时依赖 (DLLs)，确保“发给别人就能玩”。

2. 核心算法与数学公式

核心算法代码实现举例

1. 自动推进逻辑

这段代码展示了系统如何根据“是否有语音”动态决定等待时间，是播放器引擎的核心逻辑。

位置：src/player/player_window.cpp

```
void PlayerWindow::tryScheduleAutoAdvance() {
    if (!m_autoPlayEnabled || !m_autoAdvanceTimer) return;
    if (m_typewriter.isRunning()) return; // 文字还没出完，不触发计时

    const RuntimeLine &line = m_script.at(m_currentIndex);
    const QString voiceAbsPath = resolveAssetPath(line.voicePath);
    const bool hasVoice=!voiceAbsPath.isEmpty() && QFileInfo::exists(voiceAbsPath);

    if (hasVoice) {
        // 算法逻辑：若有语音，且语音正在播放，则进入等待状态，由语音播放结束信号触发推进
    }
}
```

```

if (m_voicePlayer&& m_voicePlayer->playbackState() == QMediaPlayer::PlayingState)
{
    m_waitingVoiceFinishForAutoAdvance = true;
    m_autoAdvanceTimer->stop();
    return;
}

m_autoAdvanceTimer->start(120); // 语音已播完，极短延迟后进入下一句
} else {
    // 算法逻辑：旁白或无语音对话，固定等待 2000ms
    m_waitingVoiceFinishForAutoAdvance = false;
    m_autoAdvanceTimer->start(2000);
}
}

```

2. 立绘自由定位与渲染

通过 `QGraphicsItem` 的坐标系统，实现立绘在 1920x1080 虚拟画布上的像素级精确定位。

位置：`src/player/player\_window.cpp`

```

void PlayerWindow::renderCurrentLine() {
    // ... 获取当前行数据 ...

    const RuntimeLine &line = m_script.at(m_currentIndex);

    // 渲染立绘：从数据中读取 x, y 坐标及缩放比例

    const int clampedScale = qBound(10, line.portraitScale, 300);
    const int targetHeight = qMax(60, 600 * clampedScale / 100);

    const QPixmap portraitScaled = portrait.scaledToHeight(targetHeight,
        Qt::SmoothTransformation);

    m_portraitItem->setPixmap(portraitScaled);

    // 核心算法：将逻辑坐标直接映射到场景坐标系

    m_portraitItem->setPos(line.portraitX, line.portraitY);
    m_portraitItem->setVisible(true);
    // ...
}

```

3. AI 工具使用说明

使用模型：codex

使用范围和具体内容:

1. 资源管理器面板

具体用途: 集中展示项目中引用的所有背景图、立绘和音频文件。支持通过右键菜单或按钮进行素材的添加、移除, 并能一键打开文件所在文件夹。

使用位置: 编辑器主界面左侧的“资源列表”停靠窗。

2. 剧情时间轴列表

具体用途: 将枯燥的 JSON 脚本转化为可视化的行列表。每一行显示说话人姓名和文本片段, 点击即可快速定位编辑。AI 负责了列表项的样式设计, 使其在对话较多时依然保持易读性。

使用位置: 编辑器主界面底部的“剧情流程”展示区。

3. 播放器运行时设置对话框 (In-game Settings Dialog)

具体用途: 在游戏运行过程中, 提供一个浮层界面, 允许用户通过滑动条实时调节背景音乐和语音的音量, 或选择退出游戏返回主菜单。

使用位置: 播放器窗口内, 点击右下角设置按钮后弹出的交互对话框。

4. 打字机效果渲染逻辑

具体用途: 控制对话文字按照设定的速度逐字显现, 而不是瞬间弹出。AI 实现了计时器驱动的角色渲染逻辑, 并确保在用户再次点击时能立即跳过动画显示全文本。

使用位置: 播放器画布底部的对话文本渲染层。

5. 自动播放状态指示器

具体用途: 当用户按下快捷键开启“自动播放”模式时, 在屏幕角落显示一个动态的图标 (如“▶ AUTO”), 给予用户明确的状态反馈, 防止误操作。

使用位置: 播放器窗口右上角的 HUD 悬浮显示层。

6. 导出合法性校验弹窗 (Validation Popups)

具体用途: 在执行导出前, 自动扫描整个项目。如果发现某行对话缺少立绘, 或者引用的音频文件被意外删除, AI 实现的弹窗会准确指出错误位置 (如: “第 12 行语音文件丢失”)。

使用位置: 点击“导出游戏”按钮后的预检阶段。

7. 全局按钮交互样式

具体用途: 为软件中所有的功能按钮编写 QSS (Qt 样式表), 实现鼠标悬停变色、点击反馈以及圆角边框等视觉美化, 提升软件的“精致感”。

使用位置: 编辑器与播放器的所有按钮组件 (如: 保存、导出、开始游戏等)。

五. 单元测试

在开发过程中, 我针对核心模块设计了以下功能性单元测试 (Functional Unit Tests):

测试案例 1：数据序列化与稳定性测试

测试目的：验证复杂数据（含中文、特殊路径、坐标值）在保存与读取后是否保持一致。

测试代码逻辑（参考 `src/models/model\_test.cpp`）：

```
Project p;

p.setName("测试项目#1");

Character *c = new Character();

c->setPortraitX(500); // 设置非默认坐标

p.addCharacter(c);

p.saveToFile("test.json");


Project *p2 = Project::loadFromFile("test.json");

assert(p2->name() == p.name());

assert(p2->characters()[0]->portraitX() == 500);
```

测试结果：Passed。

结果记录：生成的 `project.json` 文件格式正确，中文无乱码。读取后立绘坐标 X=500 准确恢复，验证了数据层的稳定性。

测试案例 2：资源导出兜底解析测试

测试目的：验证当用户移动了素材文件夹后，导出引擎能否通过“文件名搜索”自动修复路径。

测试环境：

1. 在项目中引用 `D:/OldPath/bg.jpg`。
2. 物理删除 `D:/OldPath/`，将文件移动至项目的 `resources/backgrounds/`。
3. 执行导出。

测试结果：Passed。

结果记录：`Exporter` 调用 `resolveExportAudioSourcePath` 成功在项目资源目录下找回了同名文件，导出包内的 `game.json` 路径更新为相对路径，游戏运行正常。

测试案例 3：自动播放状态切换测试

测试目的：验证在“快速连点”和“自动播放”交替时，系统不会崩溃且音频不会重叠。

测试操作：

1. 开启自动播放（A 键）。
2. 在语音播放到一半时，手动点击鼠标强制跳转。

测试结果：Passed。

结果记录：手动点击触发了 ``m_voicePlayer->stop()'`，旧语音立即停止，新语音正常开启。状态机 ``m_waitingVoiceFinishForAutoAdvance'` 成功重置，未出现逻辑死循环。

通过上述核心代码的严谨逻辑与多维度的单元测试，本项目实现了：

1. 高可靠性：数据存储与路径解析具备较强的容错能力。
2. 视听同步：通过状态机解决了复杂的音画同步与自动推进问题。
3. 工程化标准：通过模拟部署测试，确保了项目在不同环境下的一致性。

六. 收获

通过本次《高级语言程序设计》大作业——Galgame 可视化编辑器与播放引擎的开发，我不仅在 C++ 编程技术上有了长足进步，更在软件工程思维和前沿开发模式上有了深刻的体会。

1. 深度掌握 C++ 与 Qt 框架的实战应用

面向对象程序设计 (OOP)：通过设计 ``Project'`、``Character'`、``Dialogue'` 等核心类，我真正理解了“封装”与“继承”的意义。将复杂的剧情逻辑抽象为数据模型，实现了编辑器与播放器之间的高效解耦。

信号与槽机制：深入实践了 Qt 的事件驱动模型。在实现“属性面板实时同步预览”功能时，我深刻体会到信号槽在处理复杂 UI 交互逻辑时的优雅与高效。

多媒体处理与状态机：在开发播放引擎时，我学会了如何利用 ``QMediaPlayer'` 处理并行音频流，并通过构建状态机来管理文字打字机效果、语音同步及自动播放逻辑，这锻炼了我的逻辑严密性。

2. 软件工程思维的初步建立

模块化与解耦：项目确立了“编辑器生产 JSON，播放器解析 JSON”的架构。这种前后端分离的思路，使得两个模块可以独立开发与测试，极大地降低了系统复杂度。

部署与鲁棒性意识：在解决“导出游戏无声”和“Windows 安全拦截启动脚本”的过程中，我意识到写出代码只是第一步。真正的软件开发需要考虑资源路径的动态解析、运行时依赖 (DLLs) 的补齐以及跨环境运行的兼容性问题。

3. 人机协作编程的全新体验

开发效率的飞跃：在本次大作业中，我尝试了一种全新的开发模式：我负责底层架构设计与核心逻辑算法，AI 负责辅助生成 UI 模板与样式细节。这种分工让我能从繁琐的像素级 UI 调整中解放出来，专注于程序核心功能的实现。

从“写代码”到“架构设计”：通过与 AI 协作，我意识到未来的程序员不仅需要扎实的编码能力，更需要清晰的逻辑表达能力和对项目全局的掌控能力。学会如何精准描述需求并审核 AI 生成的代码，是我本次大作业最大的额外收获。

4. 严谨的调试与问题解决能力

项目开发并非一帆风顺。面对 VBS 脚本报错、路径编码冲突、立绘坐标偏移等棘手问

题，我学会了利用日志输出、断点调试以及查阅官方文档来定位根源。这种“发现问题、拆解问题、解决问题”的过程，极大地磨炼了我的耐心和工程实践能力。

总结：

这次大作业对我而言，不仅是一次 C++ 知识的温习，更是一次完整的、从零到一的软件产品诞生记。它让我明白，优秀的软件不仅需要精妙的算法，更需要对用户体验的关注和对工程细节的执着。在南开大学的学习生涯中，这将是我在计算机科学领域迈出的坚实一步。